

Unitree Week 16

Presentation by Mario Ortiz

This Week

More research on imitation learning with the unitree G1.

Isaac Lab Mimic

- Preface: Isaac Lab Mimic is stated to only work with Linux Operating Systems.
- Teleoperation is the primary method Isaac Lab Mimic utilizes for mimic/imitation learning.
 - Commands in “SE(2)” space (linear x-y velocity and yaw rate as outputs) and “SE(3)” space (6-D vector representing changes in pose) for robot control.
- Provides command for Inverse Kinematics Control using a keyboard device as well as a SpaceMouse.\

```
./isaacclab.sh -p scripts/environments/teleoperation/teleop_se3_agent.py  
--task Isaac-Stack-Cube-Franka-IK-Rel-v0 --num_envs 1 --teleop_device  
keyboard
```

SpaceMouse



- 3D Mouse designed for 3D navigation.
- Controller cap with 6 d.o.f
- Simultaneous panning, zooming,

Imitation Learning with Isaac Lab Mimic

- Already made scripts for collecting demonstrations from teleoperation using the “Isaac-Stack-Cube-Franka-IK-Rel-v0” environment. Can try to make a program based on this for G1.

```
# step a: create folder for datasets
mkdir -p datasets
# step b: collect data with a selected teleoperation device. Replace <teleop_device> with your preferred input device.
# Available options: spacemouse, keyboard, handtracking
./isaacclab.sh -p scripts/tools/record_demos.py --task Isaac-Stack-Cube-Franka-IK-Rel-v0 --device cpu --teleop_device <teleop_device> --dataset_file ./datasets/dataset.hdf5 --num_demos 10
# step a: replay the collected dataset
./isaacclab.sh -p scripts/tools/replay_demos.py --task Isaac-Stack-Cube-Franka-IK-Rel-v0 --device cpu --dataset_file ./datasets/dataset.hdf5
```

Imitation Learning with Isaac Lab Mimic

- It's stated that it's good to have about 10 successful demonstrations for the imitation learning workflow.
- Also stated it's better that demonstrations are short, concisely executes the task without any pauses in between (continuous motions).

This ensures the policy only focuses on the task.

Pre-recorded demonstrations

- The site also provides a pre-recorded hdf5 file containing the demonstrations for the previous example.
- Can learn more on how hdf5 files work by looking into the dataset.

Other demonstrations with Isaac Lab Mimic

- Provides some other examples (state-based policy and visuomotor policy). And shows the codes to make and view the demonstrations“
 1. First, annotate the subtasks in the recording
 2. Use Isaac Lab Mimic to generation some additional demonstrations.
 3. Inspect the output of generated data, and if satisfactory, generate the full dataset“
- Allows to increase number of demonstrations and environments.

Other demonstrations with Isaac Lab Mimic

In order to use Isaac Lab Mimic with the recorded dataset, first annotate the subtasks in the recording:

State-based policy

Visuomotor policy

```
./isaacclab.sh -p scripts/imitation_learning/isaacclab_mimic/annotate_demos.py \  
--device cpu --task Isaac-Stack-Cube-Franka-IK-Rel-Mimic-v0 --auto \  
--input_file ./datasets/dataset.hdf5 --output_file ./datasets/annotated_dataset.hdf5
```

Then, use Isaac Lab Mimic to generate some additional demonstrations:

State-based policy

Visuomotor policy

```
./isaacclab.sh -p scripts/imitation_learning/isaacclab_mimic/generate_dataset.py \  
--device cpu --num_envs 10 --generation_num_trials 10 \  
--input_file ./datasets/annotated_dataset.hdf5 --output_file ./datasets/generated_dataset_small.hdf5
```

Note

The output_file of the `annotate_demos.py` script is the input_file to the `generate_dataset.py` script

Inspect the output of generated data (filename: `generated_dataset_small.hdf5`), and if satisfactory, generate the full dataset:

State-based policy

Visuomotor policy

```
./isaacclab.sh -p scripts/imitation_learning/isaacclab_mimic/generate_dataset.py \  
--device cpu --headless --num_envs 10 --generation_num_trials 1000 \  
--input_file ./datasets/annotated_dataset.hdf5 --output_file ./datasets/generated_dataset.hdf5
```

Robomimic

Robomimic is the framework for imitation learning.

- They provide example commands for training a “BC agent” using robomimic for training a policy.

To install the robomimic framework, use the following commands:

```
# install the dependencies
sudo apt install cmake build-essential
# install python module (for robomimic)
./isaacclab.sh -i robomimic
```

State-based policy

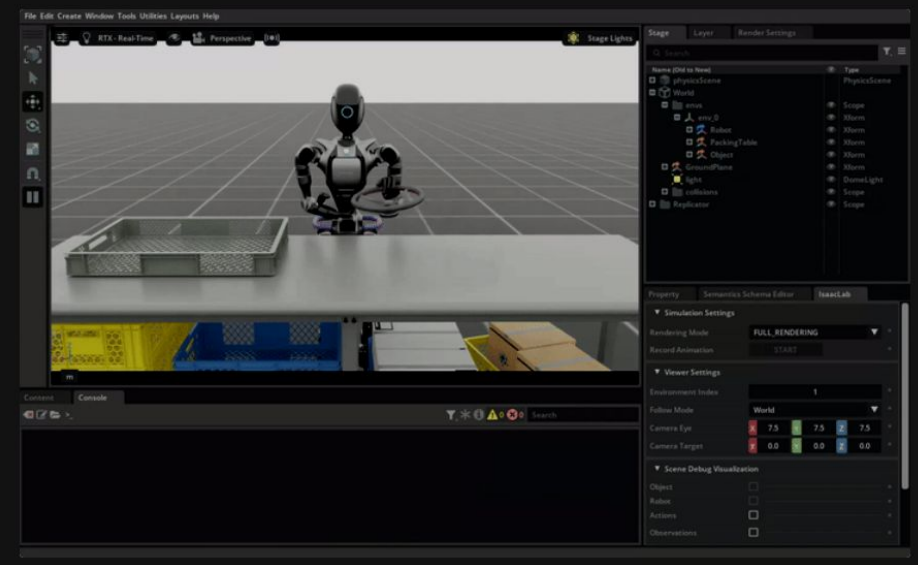
Visuomotor policy

```
./isaacclab.sh -p scripts/imitation_learning/robomimic/train.py \
--task Isaac-Stack-Cube-Franka-IK-Rel-v0 --algo bc \
--dataset ./datasets/generated_dataset.hdf5
```

Demo 1:

- A demo is provided showing Data Generation and Policy Training for a “Fourier GR-1 humanoid robot”.
- Understanding the code and process behind this demo will be useful for how we can use imitation learning to train the Unitree R1.

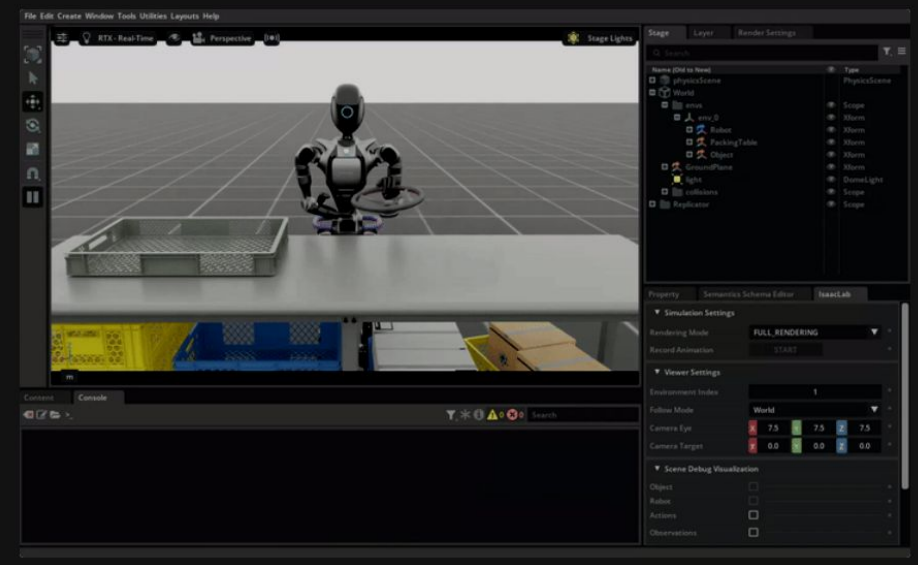
Demo 1: Data Generation and Policy Training for a Humanoid Robot



Demo 1:

- They preface on the website that “data collection for the GR-1 humanoid robot requires use of an Apple Vision pro headset”.
- There is probably cheaper alternatives to make demonstrations. **will need to research any better options for the R1.

Demo 1: Data Generation and Policy Training for a Humanoid Robot



Ideas on Collecting Teleoperation Data

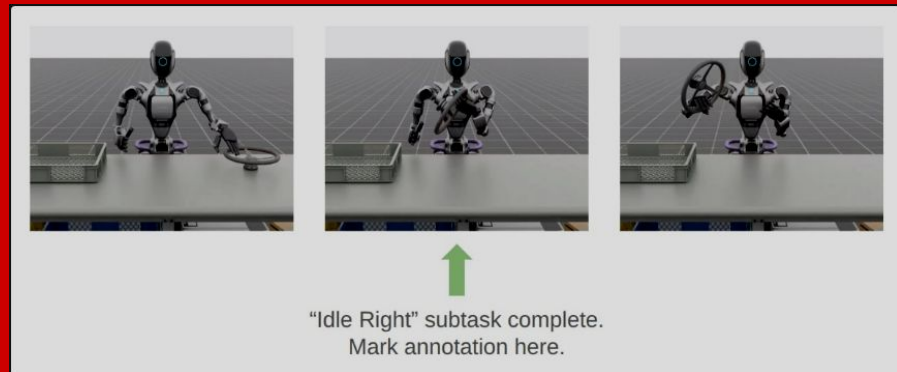
Alternative options that we can use instead of the Apple Vision Pro (or any XR headset of the sort):

- SpaceMouse: most effective for 3D manipulation, providing fluid continuous motions which are better for simulating the R1's movement.
- Gamepad (Controller such as the Playstation Controller): utilize joysticks to control the movement (SE2) or end effectors (SE3)
- Keyboard: most accessible; coordinate-based mapping to move the robot's limbs; however the movements are more binary, making it difficult to simulate realistic demos.

You can modify the scripts on the Isaac Mimic Lab site and set the teleop_device to whatever needed.

Demo 1: Annotating demonstrations

- They discuss how annotating the demonstrations is a good way to structure data, marking specific segments (such as idle, place for example).



Demo 1: Generating Dataset

- The site provides pre-recorded annotated datasets (HDF5 file) of the GR-1 robot's demonstrations.
 - (maybe we can convert them for the purpose of the R1 using digital cousin as discussed previously).
- The section provides command for generating the dataset for isaac lab.

Demo 1: Train a Policy

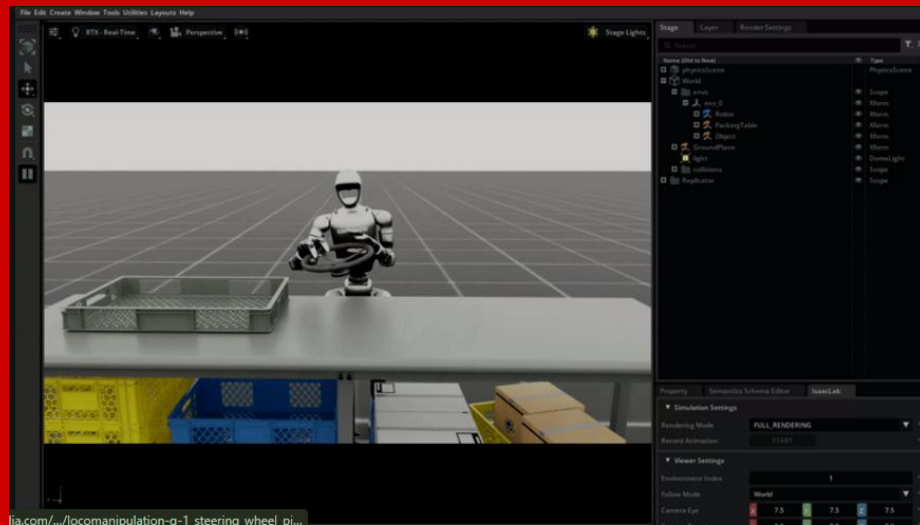
- Robomimic commands are used for training a policy based on the HDF5 dataset.

Use **Robomimic** to train a policy for the generated dataset.

```
./isaacclab.sh -p scripts/imitation_learning/robomimic/train.py \  
--task Isaac-PickPlace-GR1T2-Abs-v0 --algo bc \  
--normalize_training_actions \  
--dataset ../datasets/generated_dataset_gr1.hdf5
```

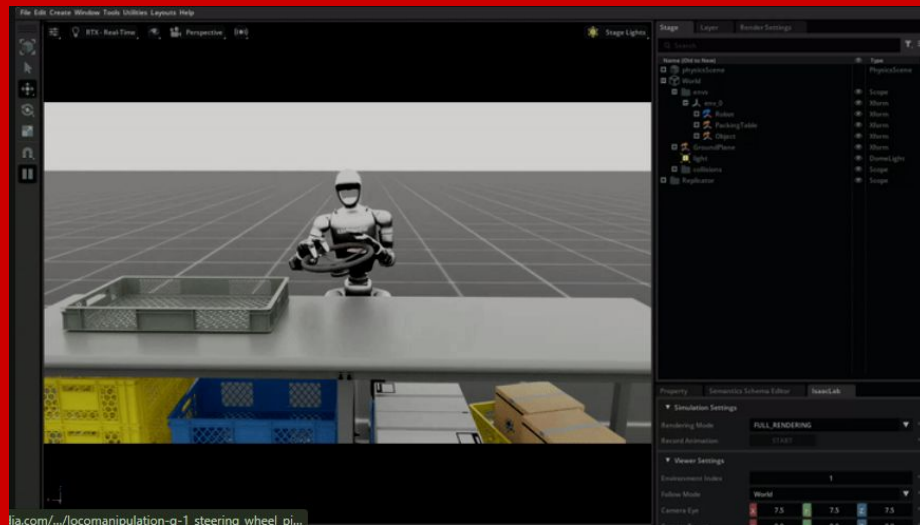
Demo 2: Data Generation and Policy Training for Humanoid Robot Locomanipulation with Unitree G1

- This demo is more important for us as it uses the Unitree G1 robot, which is very similar to the Unitree R1.
- It is different from Demo 1 as it implements a “navigation component that uses specialized scripts to generate scenes where the humanoid robot must move from point A to point B.” (Robot picks object at Point A, then places object at Point B)



Demo 2:

- Similar to the Demo 1, an HDF5 dataset has been provided for the training from the demo.



Making Unitree G1 demos

- Will be making virtual teleoperation demos
- 1. Run Isaac Lab environment on PC
- 2. Use `--teleop_device keyboard`, which should open simulator window with the virtual G1.
- 3. As you use the keyboard to move the robot joints, use the script “`record_demos.py`” to capture data, which will then be saved in the HDF5 file while done.

Making Unitree G1 demos

Command for finding IsaacLab G1 task IDs inside the source file:

```
PS C:\Users\Mario\IsaacLab> dir -Recurse -Path ".\source\" -Filter "*g1*"
```

Sections:

- Unitree
- Inspire
- Trihand
- Test_ik_configs
- Pinnochio_envs
- Locomanipulation_sdg
- Pick_pkace
- velocity

Making Unitree G1 demos

- What we can use for item pickup:

```
Directory: C:\Users\Mario\IsaacLab\source\isaacsim_tasks\isaacsim_tasks\manager_based\locomanipulation\pick_place
```

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-a----	2/12/2026 10:24 AM	8433	fixed_base_upper_body_ik_g1_env_cfg.py
-a----	2/12/2026 10:24 AM	10617	locomanipulation_g1_env_cfg.py

```
Directory: C:\Users\Mario\IsaacLab\source\isaacsim_tasks\isaacsim_tasks\manager_based\manipulation\pick_place
```

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-a----	2/12/2026 10:24 AM	15282	pickplace_unitree_g1_inspire_hand_env_cfg.py

Making Unitree G1 demos

- Next, must find the proper Task name in my IsaacLab (version 2.3.2; also installed via github repository) for demo recording.

Get-Content

```
"source\isaacclab_tasks\isaacclab_tasks\manager_based\locomanipulation  
\pick_place\__init__.py"
```

Found the specific names to implement to the command:

```
$env:PYTHONPATH =
```

```
"C:\Users\Mario\IsaacLab\source;C:\Users\Mario\IsaacLab\source\isaacclab_tasks;$env:PYTHONPATH"
```

```
.\isaacclab.bat -p scripts/tools/record_demos.py --task
```

```
Isaac-PickPlace-FixedBaseUpperBodyIK-G1-Abs-v0 --teleop_device
```

```
keyboard --num_demos 5
```

References

Teleoperation and Imitation Learning with Isaac Lab Mimic:
https://isaac-sim.github.io/IsaacLab/main/source/overview/imitation-learning/teleop_imitation.html